Magic Number: **90363**

```
AWS Keys — hadoop@ip-10-0-0-140:~ — ssh -i ~/Downloads/AWS Keys/emr-key-pair.pem hadoop@...
[hadoop@ip-10-0-0-140 ~]$ ls
TestDataGen.class
[hadoop@ip-10-0-0-140 ~]$ java TestDataGen
Magic Number = 90363
[hadoop@ip-10-0-0-140 ~]$
```

Q1. To Create Database:

```
CREATE DATABASE MyDB;
USE MyDB;
```

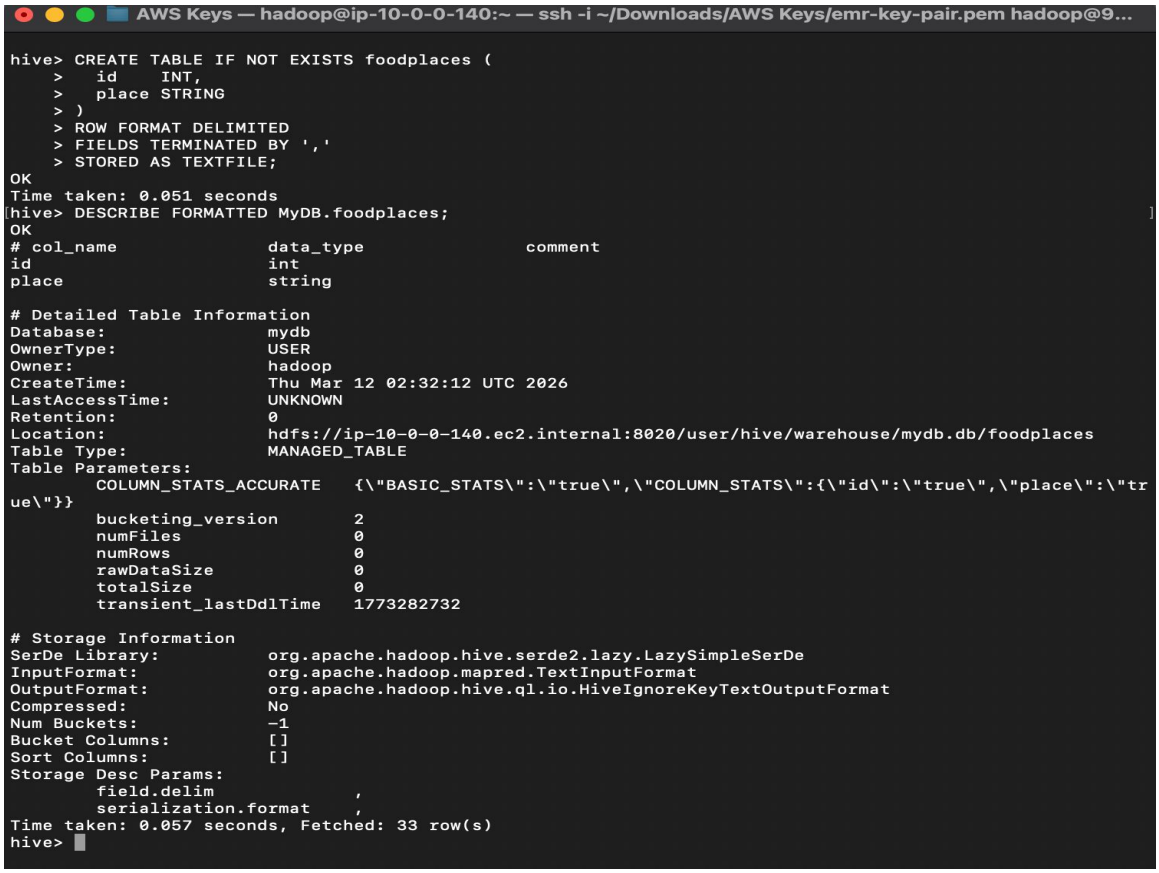
- To Create Table foodratings:

```
CREATE TABLE IF NOT EXISTS foodratings (
name  STRING COMMENT 'critic name',
food1 INT    COMMENT 'rating of food1',
food2 INT    COMMENT 'rating of food2',
food3 INT    COMMENT 'rating of food3',
food4 INT    COMMENT 'rating of food4',
id    INT    COMMENT 'restaurant id'
)
COMMENT 'ratings table stored as CSV'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

DESCRIBE FORMATTED MyDb.foodratings;
```

```
AWS Keys — hadoop@ip-10-0-0-140:~ — ssh -i ~/Downloads/AWS Keys/emr-key-pair.pem hadoop@9...
hive> CREATE TABLE IF NOT EXISTS foodratings (
    >   name  STRING COMMENT 'critic name',
    >   food1 INT    COMMENT 'rating of food1',
    >   food2 INT    COMMENT 'rating of food2',
    >   food3 INT    COMMENT 'rating of food3',
    >   food4 INT    COMMENT 'rating of food4',
    >   id    INT    COMMENT 'restaurant id'
    > )
    > COMMENT 'ratings table stored as CSV'
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > STORED AS TEXTFILE;
OK
Time taken: 0.157 seconds
hive> DESCRIBE FORMATTED MyDb.foodratings;
OK
# col_name              data_type               comment
name                    string                  critic name
food1                   int                     rating of food1
food2                   int                     rating of food2
food3                   int                     rating of food3
food4                   int                     rating of food4
id                      int                     restaurant id

# Detailed Table Information
Database:               mydb
OwnerType:              USER
Owner:                  hadoop
CreateTime:             Thu Mar 12 02:30:18 UTC 2026
LastAccessTime:         UNKNOWN
Retention:              0
Location:               hdfs://ip-10-0-0-140.ec2.internal:8020/user/hive/warehouse/mydb.db/foodratings
Table Type:             MANAGED_TABLE
Table Parameters:
        COLUMN_STATS_ACCURATE   {\"BASIC_STATS\":\"true\",\"COLUMN_STATS\":{\"food1\":\"true\",\"food2\":\
"true\",\"food3\":\"true\",\"food4\":\"true\",\"id\":\"true\",\"name\":\"true\"}}
        bucketing_version       2
        comment                 ratings table stored as CSV
        numFiles                0
        numRows                 0
        rawDataSize             0
        totalSize               0
        transient_lastDdlTime   1773282618

# Storage Information
SerDe Library:          org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe
InputFormat:            org.apache.hadoop.mapred.TextInputFormat
OutputFormat:           org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat
Compressed:             No
Num Buckets:            -1
Bucket Columns:         []
Sort Columns:           []
Storage Desc Params:
        field.delim             ,
        serialization.format    ,
Time taken: 0.157 seconds, Fetched: 38 row(s)
hive>
```

- To Create Table foodplaces:

```
CREATE TABLE IF NOT EXISTS foodplaces (
id     INT,
place STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

DESCRIBE FORMATTED MyDb.foodplaces;
```



Q2. Loading Data:

```
LOAD DATA LOCAL INPATH '/home/hadoop/foodratings90363.txt'
OVERWRITE INTO TABLE foodratings;
```

```
AWS Keys — hadoop@ip-10-0-0-140:~ — ssh -i ~/Downloads/AWS Keys/emr-key-pair.pem hadoop@9...

hive> LOAD DATA LOCAL INPATH '/home/hadoop/foodratings90363.txt' OVERWRITE INTO TABLE foodratings;
Loading data to table mydb.foodratings
OK
Time taken: 0.221 seconds
hive>
```

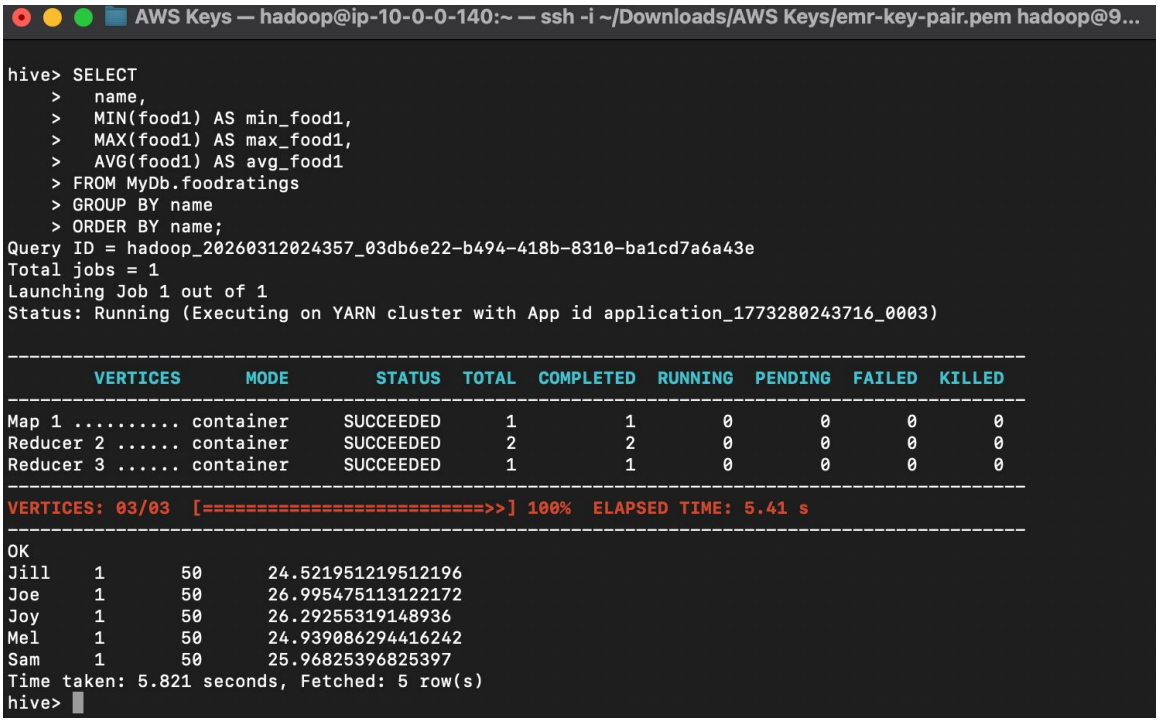
- Execute a single hive command to output the min, max and average of the values of the food3 column of the foodratings table.

```
SELECT
MIN(food3) AS min_food3,
MAX(food3) AS max_food3,
AVG(food3) AS avg_food3
FROM MyDb.foodratings;

Magic Number: 90363
```

```
AWS Keys — hadoop@ip-10-0-0-140:~ — ssh -i ~/Downloads/AWS Keys/emr-key-pair.pem hadoop@9...

hive> LOAD DATA LOCAL INPATH '/home/hadoop/foodratings90363.txt' OVERWRITE INTO TABLE foodratings;
Loading data to table mydb.foodratings
OK
Time taken: 0.221 seconds
hive> SELECT
    >   MIN(food3) AS min_food3,
    >   MAX(food3) AS max_food3,
    >   AVG(food3) AS avg_food3
    > FROM MyDb.foodratings;
Query ID = hadoop_20260312024120_6fc36e3f-42d1-442f-a83c-c23d1c35ae83
Total jobs = 1
Launching Job 1 out of 1
Tez session was closed. Reopening...
Session re-established.
Session re-established.
Status: Running (Executing on YARN cluster with App id application_1773280243716_0003)

--------------------------------------------------------------------------------
        VERTICES      MODE        STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 ..........  container     SUCCEEDED      1          1        0        0       0       0
Reducer 2 ......  container     SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 02/02  [==========================>>] 100%  ELAPSED TIME: 5.32 s
--------------------------------------------------------------------------------
OK
1       50      25.086
Time taken: 13.071 seconds, Fetched: 1 row(s)
```

Q3. Execute a hive command to output the min, max and average of the values of the food1 column grouped by the first column 'name'.

```
SELECT
name,
MIN(food1) AS min_food1,
MAX(food1) AS max_food1,
AVG(food1) AS avg_food1
FROM MyDb.foodratings
GROUP BY name
ORDER BY name;

Magic Number: 90363
```



Q4. Create a Partitioned table 'foodratingspart':

```
CREATE TABLE IF NOT EXISTS foodratingspart (
food1 INT,
food2 INT,
food3 INT,
food4 INT,
id    INT
)
PARTITIONED BY (name STRING)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

DESCRIBE FORMATTED MyDb.foodratingspart;
```

```
AWS Keys — hadoop@ip-10-0-0-140:~ — ssh -i ~/Downloads/AWS Keys/emr-key-pair.pem hadoop@9...
hive> CREATE TABLE IF NOT EXISTS foodratingspart (
    >   food1 INT,
    >   food2 INT,
    >   food3 INT,
    >   food4 INT,
    >   id    INT
    > )
    > PARTITIONED BY (name STRING)
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > STORED AS TEXTFILE;
OK
Time taken: 0.057 seconds
hive> DESCRIBE FORMATTED MyDB.foodratingspart;
OK
# col_name              data_type               comment
food1                   int
food2                   int
food3                   int
food4                   int
id                      int

# Partition Information
# col_name              data_type               comment
name                    string

# Detailed Table Information
Database:               mydb
OwnerType:              USER
Owner:                  hadoop
CreateTime:             Thu Mar 12 02:48:51 UTC 2026
LastAccessTime:         UNKNOWN
Retention:              0
Location:               hdfs://ip-10-0-0-140.ec2.internal:8020/user/hive/warehouse/mydb.db/foodratingspar
Table Type:             MANAGED_TABLE
Table Parameters:
        COLUMN_STATS_ACCURATE   {\"BASIC_STATS\":\"true\"}
        bucketing_version       2
        numFiles                0
        numPartitions           0
        numRows                 0
        rawDataSize             0
        totalSize               0
        transient_lastDdlTime   1773283731

# Storage Information
SerDe Library:          org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe
InputFormat:            org.apache.hadoop.mapred.TextInputFormat
OutputFormat:           org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat
Compressed:             No
Num Buckets:            -1
Bucket Columns:         []
Sort Columns:           []
Storage Desc Params:
        field.delim             ,
        serialization.format    ,
Time taken: 0.105 seconds, Fetched: 41 row(s)
hive>
```

Q5. Assume that the number of food critics is relatively small, say less than 10 and the number places to eat is very large, say more than 10,000. In a few short sentences explain why using the (critic) name is a good choice for a partition field while using the place id is not.

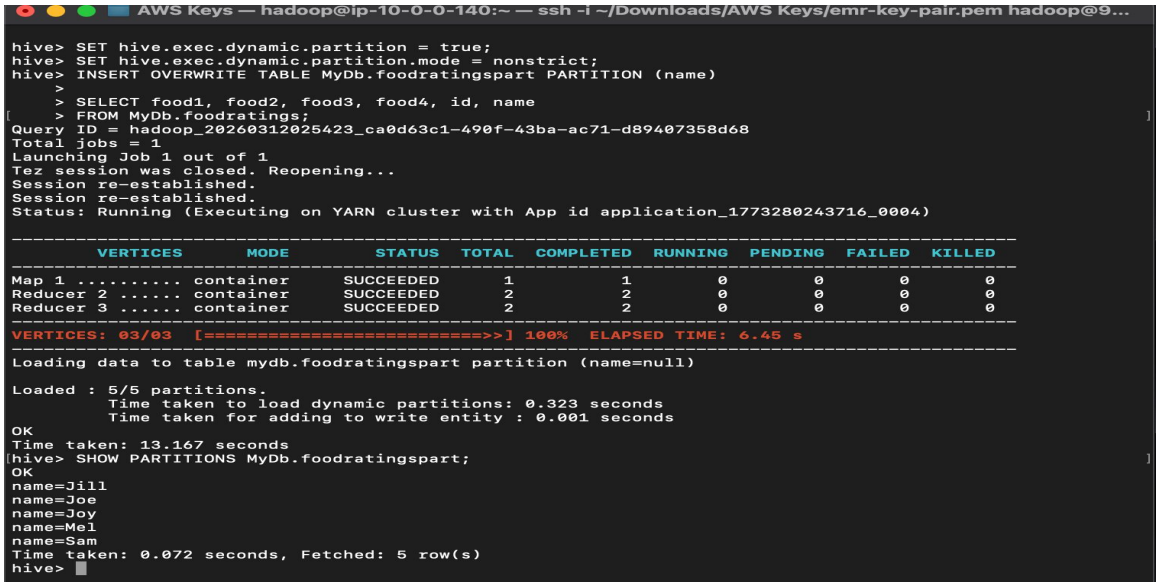
Ans:
- This is because, first of all, as the number of critic name is relatively small, partitioning the table based on name would be having relatively small number of partitions. Thus, our large number of records would be distributed under these few number of partitions.

- As against the name, if we partition our tables using place id, it will create a very large number of partitions with small chunk of data in each partition, causing Over Partitioning. It will cause an increased overhead in data loading and retrieval.

- That is the reason why, we choose (critic) name as our partitioning column.

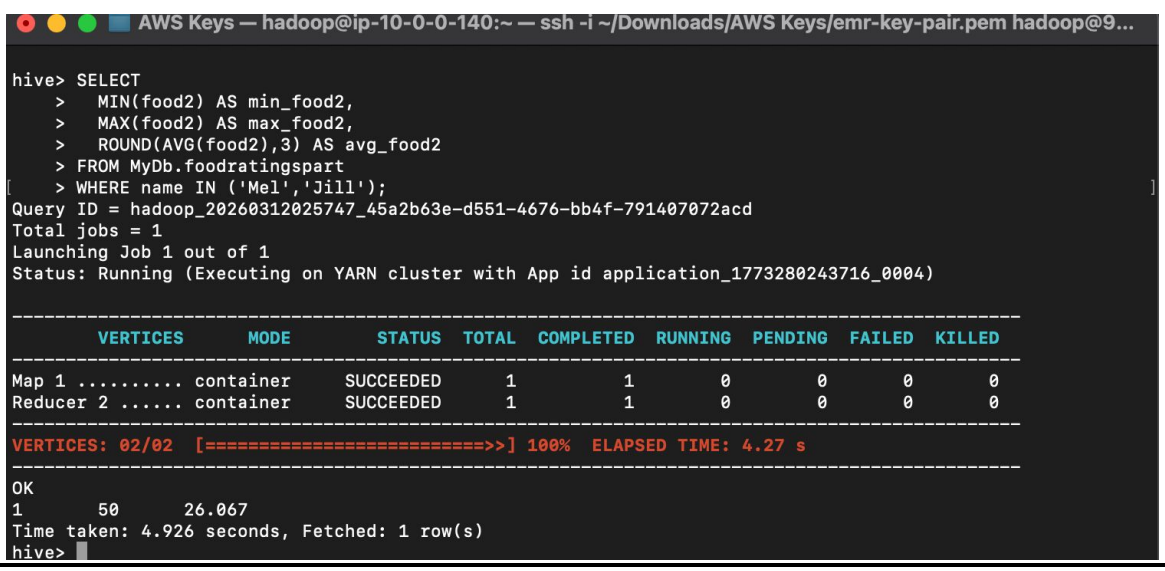
Q6. Allow dynamic partition creation and copy from MyDB.foodratings into MyDB.foodratingspart to create a partitioned table from a non-partitioned one.

```
SET hive.exec.dynamic.partition = true;
SET hive.exec.dynamic.partition.mode = nonstrict;

INSERT OVERWRITE TABLE MyDb.foodratingspart PARTITION (name)

SELECT food1, food2, food3, food4, id, name
FROM MyDb.foodratings;

SHOW PARTITIONS MyDb.foodratingspart;
```



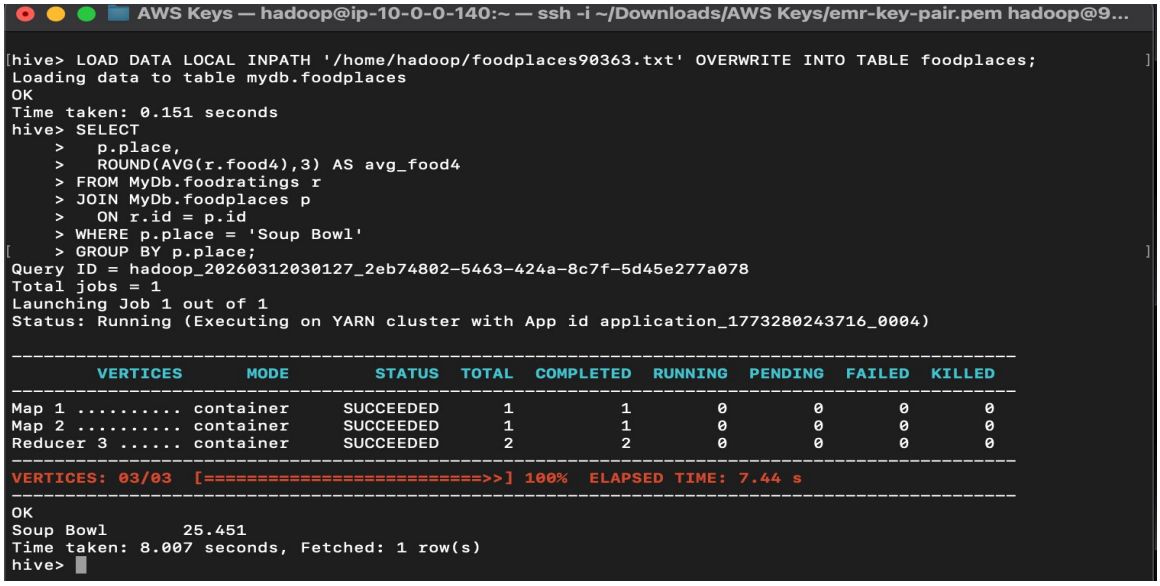
- Execute MIN, MAX, AVG on 'food2' of MyDb.foodratingspart:

```
SELECT
MIN(food2) AS min_food2,
MAX(food2) AS max_food2,
ROUND(AVG(food2),3) AS avg_food2
FROM MyDb.foodratingspart
WHERE name IN ('Mel','Jill');
```



Q7. Load the foodplaces90363.txt file created using TestDataGen from your local file system into the foodplaces table.

```
LOAD DATA LOCAL INPATH '/home/hadoop/foodplaces90363.txt'
OVERWRITE INTO TABLE foodratings;
```



```
AWS Keys — hadoop@ip-10-0-0-140:~ — ssh -i ~/Downloads/AWS Keys/emr-key-pair.pem hadoop@9...

hive> LOAD DATA LOCAL INPATH '/home/hadoop/foodplaces90363.txt' OVERWRITE INTO TABLE foodplaces;
Loading data to table mydb.foodplaces
OK
Time taken: 0.151 seconds
hive> SELECT
    >   p.place,
    >   ROUND(AVG(r.food4),3) AS avg_food4
    > FROM MyDb.foodratings r
    > JOIN MyDb.foodplaces p
    >   ON r.id = p.id
    > WHERE p.place = 'Soup Bowl'
    > GROUP BY p.place;
Query ID = hadoop_20260312030127_2eb74802-5463-424a-8c7f-5d45e277a078
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1773280243716_0004)

--------------------------------------------------------------------------------
        VERTICES      MODE        STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 .......... container     SUCCEEDED      1          1        0        0       0       0
Map 2 .......... container     SUCCEEDED      1          1        0        0       0       0
Reducer 3 ...... container     SUCCEEDED      2          2        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 03/03  [==========================>>] 100%  ELAPSED TIME: 7.44 s
--------------------------------------------------------------------------------
OK
Soup Bowl       25.451
Time taken: 8.007 seconds, Fetched: 1 row(s)
hive>
```

```
SELECT
p.place,
ROUND(AVG(r.food4),3) AS avg_food4
FROM MyDb.foodratings r
JOIN MyDb.foodplaces p
ON r.id = p.id
WHERE p.place = 'Soup Bowl'
GROUP BY p.place;
```

Q8. Read the article "An Introduction to Big Data Formats" found on the Canvas site in section "Articles" and provide short (2 to 4 sentence) answers to the following questions:

a) When is the most important consideration when choosing a row format and when a column format for your big data file?

- At the highest level, column-based storage is most useful when performing analytics queries that require only a subset of columns examined over very large datasets.
- If your queries require access to all or most of the columns of each row of data, row-based storage will be better suited to your needs.

b) What is "splitability" for a column file format and why is it important when processing large volumes of data?

- The ability of a file to be decomposed into smaller records that can be handled independently is called "Splitability".
- A column-based format will be more amenable to splitting into separate jobs if the query calculation is concerned with a single column at a time (Row Columnar).
- This means, they take a batch of rows and store that batch in columnar format. These batches then become split boundaries.
- Thus, processing large volumes of data efficiently, requires breaking the job up into parts that can be farmed out to separate processors through large-scale parallelization.

c) What can files stored in column format achieve better compression than those stored in row format?

- Columnar data can achieve better compression rates than row-based data.
- Storing values by column, with the same type next to each other, allows us to do more efficient compression on them than if we're storing rows of data.

d) Under what circumstances would it be the best choice to use the "Parquet" column file format?

- Parquet is commonly used with Apache Impala, an analytics database for Hadoop.
- Impala is designed for low latency and high concurrency queries on Hadoop.
- Parquet is especially adept at analyzing wide datasets with many columns.
- As each Parquet file contains binary data organized by "row group." For each row group, the data values are organized by column thus enabling compression benefits.
- Thus, Parquet is a good choice for read-heavy workloads.